

PROJECT SYNOPSIS: SMART JOB TRACKER

AI-Powered Recruitment Management & Online Assessment Platform

Course Code:	22CS-401 (Exploratory Project)	Domain :	Web & Mobile App / AI & ML
Team Lead:	Shivam Kumar (shivam1287.be23@chitkara.edu.in)	Group:	G15 / G16 / G17 (Select Group)
Team Roll No:	Lead Roll: [Your Roll No] M2: [Roll No], M3: [Roll No], M4: [Roll No]	Timeline:	July – December 2026

1. Problem Statement

Modern corporate recruitment is heavily burdened by administrative overhead and process inefficiencies. A single job opening frequently receives hundreds or thousands of digital applications, yet hiring managers typically spend only 6 to 10 seconds reviewing a resume initially. This brief manual screening is highly prone to human error, cognitive bias, and accidental omission of highly qualified candidates. Furthermore, candidates suffer from a complete lack of transparency, receiving no feedback or updates regarding their application progression. Lastly, recruitment operations remain fragmented across disconnected tools like spreadsheet files, personal email inboxes, and external testing websites, leading to data scattering, operational disorganization, and high time-to-hire metrics.

2. Objectives & Key Learnings

Objectives:

- **Automate Candidate Screening:** Implement NLP parsing and scoring of PDF resumes using Groq AI Cloud to immediately rate criteria alignment.
- **Centralize Recruitment Management:** Build dynamic HR and candidate pipelines to track applications from review to final selection.
- **Integrate Secure Assessment:** Create an online coding interface with automated compilation and tab-switch cheating prevention.
- **Seamless Video Calling:** Embed a direct peer-to-peer WebRTC video channel within the web interface to host remote interviews.

Key Learnings:

- Mastering full-stack client-server development using the modern MERN stack (React 19, Tailwind CSS 4, Node.js/Express, MongoDB Atlas).
- Integrating AI services via external JSON APIs and formatting strict prompt responses for automated Applicant Tracking Systems.
- Implementing browser security controls (HTML5 Visibility API) and real-time P2P stream handshakes (WebRTC signaling via DB states).

3. Functional & Non-Functional Requirements

Functional Requirements:

- **Secure Auth:** Role-based access control (HR vs. Candidate) with JWT session tokens and OTP verification.
- **Job Management:** HR dashboard allowing creation, modification, and toggling of active job listings.
- **AI Analysis:** Automated parsing of uploaded resume PDFs, returning an ATS score (0-100), key skills, and missing skill suggestions.
- **Test Assessment:** A Monaco-editor coding compiler executing user algorithms against backend tests, tracking cheating warnings.
- **Meeting Room:** Integrated WebRTC video room allowing live client calls and concurrent text chat.
- **Doc Generation:** Automatically creates dynamic, professional job offer letters in PDF format.

Non-Functional Requirements:

- **Data Protection:** Cryptographic encryption of user passwords using bcrypt, with API routes validated by JWT auth middleware.
- **Performance:** Execution times for file uploads and AI parsing queries kept under 3 seconds using asynchronous processes.
- **Scalability:** Modular Express router file layout and document-oriented MongoDB Atlas clustering to handle high-frequency applicant traffic.
- **Responsiveness:** Interactive visual dashboard styled with fluid CSS grid layouts fitting desktops, tablets, and phones.

4. High-Level and Low-Level Design

High-Level Design (HLD):

The system follows a clean, decoupled three-tier MVC architecture. The client (React 19 SPA) renders the visual panels and communicates via Axios HTTP requests to the backend server (Node.js & Express). The server coordinates business operations, querying the document database (MongoDB Atlas) for structured state storage, and dispatching text queries to external services. Specifically, resume files are extracted in the backend via pdf-parse, formatted, and passed to the Groq Cloud SDK running Llama-3.3-70b-versatile, which returns structured analysis JSON objects. Real-time video meetings use WebRTC with signals passed through specialized database state markers, avoiding complex external signaling servers.

Low-Level Design (LLD) & Data Modeling:

The backend data model relies on five core schemas in MongoDB, which manage the recruitment lifecycle. These collections maintain reference relations via ObjectId fields to keep state consistency:

Model Name	Key Attributes / Database Schema Details
User Schema	name, email, password (hashed), role (candidate/hr/admin), skills, education[], experience[]
Job Schema	company_name, job_role, description, required_skills, rounds, created_by (Ref: User), status (open/closed)
Application Schema	user_id (Ref: User), job_id (Ref: Job), resume_path, ats_score, ats_explanation, current_round, status
Test Schema	job_id (Ref: Job), round_number, title, questions [{question_type, details, test_cases[]}]
TestSubmission	test_id (Ref: Test), user_id (Ref: User), application_id (Ref: Application), answers[], tab_switches, total_score

5. Advantages & Disadvantages

Advantages:

- **Automated Efficiency:** slashes time-to-hire by auto-ranking resumes using an objective, criteria-based match score.
- **Unified Pipeline:** merges job postings, testing, cheating reports, remote interviews, and offer generation into one portal.
- **Constructive Feedback:** gives candidate accounts detailed recommendations on missing skills and keyword optimizations.
- **Academic Integrity:** logs tab-switching counts during technical tests, providing recruiters an objective warning system.

Disadvantages:

- **API Availability:** depends entirely on external API connections (Groq API, database hosting); network failures block core functions.
- **Bypassable Cheating Detection:** client visibility changes track active tab loss, but cannot detect off-screen resources (phones, dual monitors).
- **WebRTC Firewall Limits:** direct peer connections fail on restricted network types without TURN/STUN relay configurations.

6. Conclusion, Future Improvements & Potential Bottlenecks

Conclusion:

The Smart Job Tracker successfully constructs an automated, transparent, and cohesive recruitment environment. By replacing fragmented tools with an intelligent, MERN-stack portal, it removes the repetitive strain of manual resume screening and introduces security in applicant assessment, making it an excellent fit for the EP curriculum.

Future Improvements:

- **Blockchain Verification:** Store and verify university degree certificates on an immutable blockchain ledger to eliminate candidate fraud.
- **Video Interview Analysis:** Integrate speech-to-text transcription and facial sentiment analysis to automatically evaluate soft skills.
- **Advanced STUN/TURN:** Implement a media server layout (SFU) to scale interview rooms to multi-party panels securely.

Potential Bottlenecks:

- **Groq Rate Limits & LLM Latency:** High applicant volume could throttle API keys, causing delays in return parsing results (mitigation: job queuing).
- **Resume File Failures:** Extremely large resume sizes or non-standard PDF formats can fail parser extraction (mitigation: size caps and raw text input fallbacks).

7. References

1. React 19 Development Guide: <https://react.dev/blog/2024/12/05/react-19>
2. Express.js REST API Routing Standards: <https://expressjs.com/>
3. Groq SDK Cloud Model Usage (Llama 3.3): <https://console.groq.com/docs>
4. WebRTC Peer-to-Peer Protocol Standard Specifications: <https://webrtc.org/>
5. MongoDB Atlas Document Database Relations: <https://www.mongodb.com/docs/>